# AEDT Stage A Grounding Benchmark 实现计划

> **面向 AI 代理的工作者：** 必需子技能：使用 superpowers:subagent-driven-development（推荐）或 superpowers:executing-plans 逐任务实现此计划。步骤使用复选框（`- [ ]`）语法来跟踪进度。

**目标：** 构建 HFSS-only Stage A 离线评估基座：API 语义库（Top 50）、结构化案例库（10 个）、common traps（7 个）、节点 catalog（8 个）、Benchmark 数据集（30 个任务）、Group A/B/C 离线判卷 runner（含 semantic-lite 和 Go/No-Go 判定）、LLM 代码生成接口。

**架构：** Stage A 不连接真实 AEDT，不实现 MCP Server。它建立轻量融合实验闭环：读取 YAML/JSONL 数据资产，加载 Top 50 API 语义，组装 Group A/B/C 上下文，对生成代码做 syntax/API/security/semantic-lite 五层离线判卷，计算 Go/No-Go 指标，并输出报告。后续 Stage B 将复用本计划创建的数据结构和 Provider 接口。

**技术栈：** Python 3.11+、标准库 `sqlite3`/`ast`/`json`/`dataclasses`、PyYAML、pytest。

---

## 文件结构

创建以下文件：

```
pyproject.toml
src/aedt_agent/__init__.py
src/aedt_agent/knowledge/__init__.py
src/aedt_agent/knowledge/models.py
src/aedt_agent/knowledge/provider_interface.py
src/aedt_agent/knowledge/sqlite_provider.py
src/aedt_agent/knowledge/build_sqlite.py
src/aedt_agent/knowledge/extract_api_semantics.py
src/aedt_agent/nodes/__init__.py
src/aedt_agent/nodes/models.py
src/aedt_agent/nodes/registry.py
src/aedt_agent/benchmark/__init__.py
src/aedt_agent/benchmark/models.py
src/aedt_agent/benchmark/graders.py
src/aedt_agent/benchmark/semantic_lite.py
src/aedt_agent/benchmark/context_builder.py
src/aedt_agent/benchmark/generator.py
src/aedt_agent/benchmark/prompt_templates.py
src/aedt_agent/benchmark/repair.py
src/aedt_agent/benchmark/go_nogo.py
src/aedt_agent/benchmark/node_readiness.py
src/aedt_agent/benchmark/runner.py
src/aedt_agent/cli.py
knowledge/api_semantics/api_semantics.schema.sql
knowledge/api_semantics/api_semantics.seed.jsonl
knowledge/workflow_cases/hfss_patch_antenna.yaml
knowledge/workflow_cases/microstrip_line.yaml
knowledge/workflow_cases/rectangular_waveguide.yaml
```

```
knowledge/workflow_cases/waveguide_filter.yaml
knowledge/workflow_cases/coaxial_feed.yaml
knowledge/workflow_cases/cavity_resonator.yaml
knowledge/workflow_cases/simple_sparameter_export.yaml
knowledge/workflow_cases/differential_pair.yaml
knowledge/workflow_cases/antenna_array.yaml
knowledge/workflow_cases/vivaldi_antenna.yaml
knowledge/workflow_cases/cpw_line.yaml
knowledge/common_traps/waveport_no_background_contact.yaml
knowledge/common_traps/airbox_too_small.yaml
knowledge/common_traps/missing_ground_plane.yaml
knowledge/common_traps/wrong_face_selected_for_port.yaml
knowledge/common_traps/sweep_range_misses_target_frequency.yaml
knowledge/common_traps/material_or_unit_mismatch.yaml
knowledge/common_traps/boundary_assigned_to_wrong_object.yaml
nodes/catalog/create_substrate.yaml
nodes/catalog/create_conductor_or_geometry_group.yaml
nodes/catalog/select_face.yaml
nodes/catalog/create_port.yaml
nodes/catalog/create_airbox.yaml
nodes/catalog/assign_boundary.yaml
nodes/catalog/create_setup.yaml
nodes/catalog/create_sweep_or_export.yaml
benchmarks/tasks/L1_create_substrate.yaml
benchmarks/tasks/L1_create_conductor.yaml
benchmarks/tasks/L1_select_face.yaml
benchmarks/tasks/L1_create_wave_port.yaml
benchmarks/tasks/L1_create_lumped_port.yaml
benchmarks/tasks/L1_create_airbox.yaml
benchmarks/tasks/L1_assign_radiation_boundary.yaml
benchmarks/tasks/L1_create_setup.yaml
benchmarks/tasks/L1_create_sweep.yaml
benchmarks/tasks/L1_assign_material.yaml
benchmarks/tasks/L2_microstrip_line.yaml
benchmarks/tasks/L2_stripline.yaml
benchmarks/tasks/L2_cpw_line.yaml
benchmarks/tasks/L2_coaxial_cable.yaml
benchmarks/tasks/L2_waveguide_section.yaml
benchmarks/tasks/L2_patch_with_probe_feed.yaml
benchmarks/tasks/L2_dipole_antenna.yaml
benchmarks/tasks/L2_simple_filter.yaml
benchmarks/tasks/L2_cavity_resonator.yaml
benchmarks/tasks/L2_differential_pair.yaml
benchmarks/tasks/L3_patch_antenna_sparameter.yaml
benchmarks/tasks/L3_waveguide_filter.yaml
benchmarks/tasks/L3_antenna_array.yaml
benchmarks/tasks/L3_vivaldi_antenna.yaml
benchmarks/tasks/L3_sparameter_export.yaml
benchmarks/tasks/Trap_waveport_wrong_face.yaml
benchmarks/tasks/Trap_airbox_too_small.yaml
benchmarks/tasks/Trap_missing_ground.yaml
benchmarks/tasks/Trap_sweep_misses_freq.yaml
benchmarks/tasks/Trap_unit_mismatch.yaml
```

```
benchmarks/reference_scripts/.gitkeep
benchmarks/validation_scripts/.gitkeep
benchmarks/generated/group_a/.gitkeep
benchmarks/generated/group_b/.gitkeep
benchmarks/generated/group_c/.gitkeep
benchmarks/reports/.gitkeep
docs/validation-script-spec.md
tests/test_knowledge_provider.py
tests/test_node_registry.py
tests/test_graders.py
tests/test_semantic_lite.py
tests/test_context_builder.py
tests/test_generator.py
tests/test_repair.py
tests/test_go_nogo.py
tests/test_node_readiness.py
tests/test_api_semantics_coverage.py
tests/test_runner.py
```

职责边界：

- `knowledge/*`：只负责 API/case/trap 数据加载与检索，不知道 Benchmark 分组。
- `nodes/*`：只负责节点 YAML 加载、schema 校验和 API 白名单提取。
- `benchmark/*`：负责任务加载、上下文构建、LLM 代码生成、代码判卷（syntax/API/security/semantic-lite）、修复循环、Go/No-Go 判定和报告输出。
- `cli.py`：只提供命令入口，不包含业务逻辑。

---

## 任务 1：创建 Python 项目骨架

**文件：**

- 创建：`pyproject.toml`
- 创建：`src/aedt_agent/__init__.py`
- 创建：`src/aedt_agent/knowledge/__init__.py`
- 创建：`src/aedt_agent/nodes/__init__.py`
- 创建：`src/aedt_agent/benchmark/__init__.py`
- 创建：`benchmarks/reports/.gitkeep`
- 创建：`benchmarks/reference_scripts/.gitkeep`
- 创建：`benchmarks/validation_scripts/.gitkeep`
- 创建：`benchmarks/generated/group_a/.gitkeep`
- 创建：`benchmarks/generated/group_b/.gitkeep`

- 创建：`benchmarks/generated/group_c/.gitkeep`
- ☐ **步骤 1：编写项目配置**

创建 `pyproject.toml`：

```
[build-system]
requires = ["setuptools>=69", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "aedt-agent"
version = "0.1.0"
description = "Stage A grounding benchmark for AEDT node simulation agent"
requires-python = ">=3.11"
dependencies = [
    "PyYAML>=6.0.1",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
]

[project.scripts]
aedt-agent = "aedt_agent.cli:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
```

- ☐ **步骤 2：创建包初始化文件**

创建 `src/aedt_agent/__init__.py`：

```
"""AEDT Agent Stage A grounding benchmark package."""

__all__ = ["__version__"]
__version__ = "0.1.0"
```

创建 `src/aedt_agent/knowledge/__init__.py`：

```
"""Knowledge providers for API semantics, workflow cases, and traps."""
```

创建 src/aedt_agent/nodes/__init__.py：

```
"""Node catalog models and registry."""
```

创建 src/aedt_agent/benchmark/__init__.py：

```
"""Benchmark task loading, context assembly, grading, and reporting."""
```

创建空 .gitkeep 文件。

- ☐ **步骤 3：运行基础测试命令确认当前没有测试**

运行：

```
python -m pytest -q
```

预期：pytest 可启动，显示没有测试或 0 个测试。

- ☐ **步骤 4：Commit**

```
git add pyproject.toml src benchmarks
git commit -m "chore: scaffold stage a benchmark package"
```

---

## 任务 2：定义知识层数据模型

**文件：**

- 创建：src/aedt_agent/knowledge/models.py
- 创建：tests/test_knowledge_provider.py
- ☐ **步骤 1：编写失败测试**

创建 tests/test_knowledge_provider.py：

```
from aedt_agent.knowledge.models import ApiSemantic, CommonTrap, WorkflowCase


def test_api_semantic_from_dict_parses_lists():
    item = ApiSemantic.from_dict(
        {
            "fqname": "Hfss.modeler.create_box",
            "domain": "hfss",
```

```
            "category": "geometry",
            "signature": "create_box(origin, sizes, name=None, material=None)",
            "params": [{"name": "origin"}],
            "returns": {"type": "ObjectId"},
            "docstring": "Create a box.",
            "constraints": ["sizes must be positive"],
            "common_errors": ["negative size"],
            "common_traps": ["unit mismatch"],
            "examples_ref": ["hfss_patch_antenna"],
            "source_refs": ["manual"],
            "confidence": "manual",
            "pyaedt_version": "0.0",
            "aedt_version": "2025R2",
            "last_verified_at": "2026-05-08",
        }
    )

    assert item.fqname == "Hfss.modeler.create_box"
    assert item.constraints == ["sizes must be positive"]
    assert item.params[0]["name"] == "origin"


def test_workflow_case_from_dict_has_steps():
    case = WorkflowCase.from_dict(
        {
            "case_id": "hfss_patch_antenna",
            "domain": "hfss",
            "task_type": "antenna",
            "natural_language_task": "Create patch antenna",
            "workflow_steps": ["create_substrate", "create_port"],
            "api_used": ["Hfss.modeler.create_box"],
            "parameters": {"frequency": "2.4GHz"},
            "reference_script":
"benchmarks/reference_scripts/hfss_patch_antenna.py",
            "validation_script":
"benchmarks/validation_scripts/validate_hfss_patch_antenna.py",
            "expected_state": {"objects": ["substrate"]},
            "known_traps": ["missing_ground_plane"],
            "notes": "Structured case.",
        }
    )

    assert case.workflow_steps == ["create_substrate", "create_port"]
    assert case.parameters["frequency"] == "2.4GHz"


def test_common_trap_from_dict_has_detection():
    trap = CommonTrap.from_dict(
        {
            "trap_id": "airbox_too_small",
            "domain": "hfss",
            "applies_to": ["create_airbox"],
            "symptom": "Radiation result is wrong",
```

```
            "root_cause": "Padding too small",
            "why_silent": "Model can solve but boundary is poor",
            "detection": "Check padding against wavelength",
            "prevention": "Use frequency-aware padding",
            "validation_rule": "validate_airbox_padding",
            "source": "manual",
        }
    )

    assert trap.trap_id == "airbox_too_small"
    assert trap.validation_rule == "validate_airbox_padding"
```

- ☐ **步骤 2：运行测试验证失败**

运行：

```
python -m pytest tests/test_knowledge_provider.py -q
```

预期：FAIL，报错包含 `ModuleNotFoundError` 或 `ImportError`。

- ☐ **步骤 3：实现数据模型**

创建 `src/aedt_agent/knowledge/models.py`：

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any


def _list_value(data: dict[str, Any], key: str) -> list[Any]:
    value = data.get(key, [])
    if value is None:
        return []
    if isinstance(value, list):
        return value
    raise TypeError(f"{key} must be a list")


def _dict_value(data: dict[str, Any], key: str) -> dict[str, Any]:
    value = data.get(key, {})
    if value is None:
        return {}
    if isinstance(value, dict):
        return value
    raise TypeError(f"{key} must be a dict")


@dataclass(frozen=True)
```

```
class ApiSemantic:
    fqname: str
    domain: str
    category: str
    signature: str
    params: list[dict[str, Any]] = field(default_factory=list)
    returns: dict[str, Any] = field(default_factory=dict)
    docstring: str = ""
    constraints: list[str] = field(default_factory=list)
    common_errors: list[str] = field(default_factory=list)
    common_traps: list[str] = field(default_factory=list)
    examples_ref: list[str] = field(default_factory=list)
    source_refs: list[str] = field(default_factory=list)
    confidence: str = "inferred"
    pyaedt_version: str = ""
    aedt_version: str = ""
    last_verified_at: str = ""

    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> "ApiSemantic":
        return cls(
            fqname=str(data["fqname"]),
            domain=str(data.get("domain", "hfss")),
            category=str(data["category"]),
            signature=str(data.get("signature", "")),
            params=_list_value(data, "params"),
            returns=_dict_value(data, "returns"),
            docstring=str(data.get("docstring", "")),
            constraints=[str(item) for item in _list_value(data,
"constraints")],
            common_errors=[str(item) for item in _list_value(data,
"common_errors")],
            common_traps=[str(item) for item in _list_value(data,
"common_traps")],
            examples_ref=[str(item) for item in _list_value(data,
"examples_ref")],
            source_refs=[str(item) for item in _list_value(data,
"source_refs")],
            confidence=str(data.get("confidence", "inferred")),
            pyaedt_version=str(data.get("pyaedt_version", "")),
            aedt_version=str(data.get("aedt_version", "")),
            last_verified_at=str(data.get("last_verified_at", "")),
        )


@dataclass(frozen=True)
class WorkflowCase:
    case_id: str
    domain: str
    task_type: str
    natural_language_task: str
    workflow_steps: list[str]
    api_used: list[str]
```

```
    parameters: dict[str, Any]
    reference_script: str
    validation_script: str
    expected_state: dict[str, Any]
    known_traps: list[str]
    notes: str = ""

    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> "WorkflowCase":
        return cls(
            case_id=str(data["case_id"]),
            domain=str(data.get("domain", "hfss")),
            task_type=str(data["task_type"]),
            natural_language_task=str(data["natural_language_task"]),
            workflow_steps=[str(item) for item in _list_value(data,
"workflow_steps")],
            api_used=[str(item) for item in _list_value(data, "api_used")],
            parameters=_dict_value(data, "parameters"),
            reference_script=str(data["reference_script"]),
            validation_script=str(data["validation_script"]),
            expected_state=_dict_value(data, "expected_state"),
            known_traps=[str(item) for item in _list_value(data,
"known_traps")],
            notes=str(data.get("notes", "")),
        )


@dataclass(frozen=True)
class CommonTrap:
    trap_id: str
    domain: str
    applies_to: list[str]
    symptom: str
    root_cause: str
    why_silent: str
    detection: str
    prevention: str
    validation_rule: str
    source: str

    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> "CommonTrap":
        return cls(
            trap_id=str(data["trap_id"]),
            domain=str(data.get("domain", "hfss")),
            applies_to=[str(item) for item in _list_value(data, "applies_to")],
            symptom=str(data["symptom"]),
            root_cause=str(data["root_cause"]),
            why_silent=str(data["why_silent"]),
            detection=str(data["detection"]),
            prevention=str(data["prevention"]),
            validation_rule=str(data["validation_rule"]),
```

```
            source=str(data["source"]),
        )
```

- ☐ **步骤 4：运行测试验证通过**

运行：

```
python -m pytest tests/test_knowledge_provider.py -q
```

预期：PASS，3 个测试通过。

- ☐ **步骤 5：Commit**

```
git add src/aedt_agent/knowledge/models.py tests/test_knowledge_provider.py
git commit -m "feat: add knowledge data models"
```

---

## 任务 3：创建 API 语义库 Schema

**文件：**

- 创建：`knowledge/api_semantics/api_semantics.schema.sql`
- 创建：`knowledge/api_semantics/api_semantics.seed.jsonl`
- ☐ **步骤 1：编写 SQLite Schema**

创建 `knowledge/api_semantics/api_semantics.schema.sql`：

```
CREATE TABLE IF NOT EXISTS api_semantics (
    fqname              TEXT PRIMARY KEY,
    domain              TEXT,
    category            TEXT,
    signature           TEXT,
    params_json         TEXT,
    returns_json        TEXT,
    docstring           TEXT,
    constraints_json    TEXT,
    common_errors_json  TEXT,
    common_traps_json   TEXT,
    examples_ref_json   TEXT,
    source_refs_json    TEXT,
    confidence          TEXT,
    pyaedt_version      TEXT,
    aedt_version        TEXT,
    last_verified_at    TEXT
);
```

```
CREATE VIRTUAL TABLE IF NOT EXISTS api_semantics_fts USING fts5(
    fqname,
    domain,
    category,
    signature,
    docstring,
    content=api_semantics,
    content_rowid=rowid
);

CREATE TRIGGER IF NOT EXISTS api_semantics_ai AFTER INSERT ON api_semantics
BEGIN
    INSERT INTO api_semantics_fts(rowid, fqname, domain, category, signature,
docstring)
    VALUES (new.rowid, new.fqname, new.domain, new.category, new.signature,
new.docstring);
END;

CREATE TRIGGER IF NOT EXISTS api_semantics_ad AFTER DELETE ON api_semantics
BEGIN
    INSERT INTO api_semantics_fts(api_semantics_fts, rowid, fqname, domain,
category, signature, docstring)
    VALUES ('delete', old.rowid, old.fqname, old.domain, old.category,
old.signature, old.docstring);
END;
```

- ☐ **步骤 2：创建初始 seed 数据**

创建 `knowledge/api_semantics/api_semantics.seed.jsonl`，包含 5 条初始数据（后续任务 9 会扩展到 Top 50）：

```
{"fqname":"Hfss.modeler.create_box","domain":"hfss","category":"geometry","sign
ature":"create_box(origin, sizes, name=None, material=None)","params_json":"
[{\"name\":\"origin\",\"type\":\"list[float]\"},
{\"name\":\"sizes\",\"type\":\"list[float]\"},
{\"name\":\"name\",\"type\":\"str\"},
{\"name\":\"material\",\"type\":\"str\"}]","returns_json":"
{\"type\":\"ObjectId\"}","docstring":"Create a 3D box in the HFSS
modeler.","constraints_json":"[\"sizes must be positive\",\"origin is [x, y, z]
in model units\"]","common_errors_json":"[\"negative size
values\"]","common_traps_json":"[\"unit_mismatch\"]","examples_ref_json":"
[\"hfss_patch_antenna\"]","source_refs_json":"
[\"manual\"]","confidence":"manual","pyaedt_version":"0.26.3","aedt_version":"2
025R2","last_verified_at":"2026-05-08"}
{"fqname":"Hfss.assign_material","domain":"hfss","category":"material","signatu
re":"assign_material(assignment, material)","params_json":"
[{\"name\":\"assignment\",\"type\":\"str\"},
{\"name\":\"material\",\"type\":\"str\"}]","returns_json":"
{\"type\":\"bool\"}","docstring":"Assign a material to an
```

```
object.","constraints_json":"[\"material must exist in material library or be
creatable\"]","common_errors_json":"[\"material not
found\"]","common_traps_json":"
[\"material_or_unit_mismatch\"]","examples_ref_json":"
[\"hfss_patch_antenna\"]","source_refs_json":"
[\"manual\"]","confidence":"manual","pyaedt_version":"0.26.3","aedt_version":"2
025R2","last_verified_at":"2026-05-08"}
{"fqname":"Hfss.modeler.create_rectangle","domain":"hfss","category":"geometry"
,"signature":"create_rectangle(origin, sizes, name=None,
material=None)","params_json":"[{\"name\":\"origin\",\"type\":\"list[float]\"},
{\"name\":\"sizes\",\"type\":\"list[float]\"},
{\"name\":\"name\",\"type\":\"str\"},
{\"name\":\"material\",\"type\":\"str\"}]","returns_json":"
{\"type\":\"ObjectId\"}","docstring":"Create a 2D rectangle sheet in the HFSS
modeler.","constraints_json":"[\"sizes must be
positive\"]","common_errors_json":"[]","common_traps_json":"
[\"unit_mismatch\"]","examples_ref_json":"
[\"hfss_patch_antenna\"]","source_refs_json":"
[\"manual\"]","confidence":"manual","pyaedt_version":"0.26.3","aedt_version":"2
025R2","last_verified_at":"2026-05-08"}
{"fqname":"Hfss.create_wave_port","domain":"hfss","category":"excitation","sign
ature":"create_wave_port(assignment, reference=None,
name=None)","params_json":"[{\"name\":\"assignment\",\"type\":\"str|FaceId\"},
{\"name\":\"reference\",\"type\":\"str\"},
{\"name\":\"name\",\"type\":\"str\"}]","returns_json":"
{\"type\":\"PortId\"}","docstring":"Create a wave port on a
face.","constraints_json":"[\"port face must touch the
background/airbox\",\"must be an exterior face\"]","common_errors_json":"
[\"face does not touch background\"]","common_traps_json":"
[\"waveport_no_background_contact\",\"wrong_face_selected_for_port\"]","example
s_ref_json":"[\"rectangular_waveguide\"]","source_refs_json":"
[\"manual\"]","confidence":"manual","pyaedt_version":"0.26.3","aedt_version":"2
025R2","last_verified_at":"2026-05-08"}
{"fqname":"Hfss.create_setup","domain":"hfss","category":"setup","signature":"c
reate_setup(name='Setup1', setup_type=None, **kwargs)","params_json":"
[{\"name\":\"name\",\"type\":\"str\"},
{\"name\":\"setup_type\",\"type\":\"str\"}]","returns_json":"
{\"type\":\"SetupId\"}","docstring":"Create an HFSS driven modal solution
setup.","constraints_json":"[\"at least one excitation must exist before
solving\"]","common_errors_json":"[\"invalid frequency
string\"]","common_traps_json":"[]","examples_ref_json":"
[\"hfss_patch_antenna\"]","source_refs_json":"
[\"manual\"]","confidence":"manual","pyaedt_version":"0.26.3","aedt_version":"2
025R2","last_verified_at":"2026-05-08"}
```


- ☐ **步骤 3：Commit**

```
git add knowledge/api_semantics
git commit -m "feat: add api semantics schema and seed data"
```

## 任务 4：实现 KnowledgeProvider 接口

**文件：**

- 创建：`src/aedt_agent/knowledge/provider_interface.py`
- ☐ **步骤 1：实现接口**

创建 `src/aedt_agent/knowledge/provider_interface.py`：

```
from __future__ import annotations

from typing import Protocol

from aedt_agent.knowledge.models import ApiSemantic, CommonTrap, WorkflowCase


class KnowledgeProvider(Protocol):
    def search_api(self, query: str, limit: int = 10) -> list[ApiSemantic]: ...
    def list_workflow_cases(self) -> list[WorkflowCase]: ...
    def list_common_traps(self, filter_ids: list[str] | None = None) ->
list[CommonTrap]: ...
```

- ☐ **步骤 2：Commit**

```
git add src/aedt_agent/knowledge/provider_interface.py
git commit -m "feat: add knowledge provider interface"
```

---

## 任务 5：实现 SQLite Provider 和构建工具

**文件：**

- 创建：`src/aedt_agent/knowledge/sqlite_provider.py`
- 创建：`src/aedt_agent/knowledge/build_sqlite.py`
- 创建 YAML 数据文件：
  - `knowledge/workflow_cases/hfss_patch_antenna.yaml`
  - `knowledge/workflow_cases/microstrip_line.yaml`
  - `knowledge/workflow_cases/rectangular_waveguide.yaml`
  - `knowledge/common_traps/waveport_no_background_contact.yaml`
  - `knowledge/common_traps/airbox_too_small.yaml`
  - `knowledge/common_traps/missing_ground_plane.yaml`
- 修改：`tests/test_knowledge_provider.py`（追加测试）

- ☐ **步骤 1：追加失败测试**

在 `tests/test_knowledge_provider.py` 末尾追加：

```
from pathlib import Path
from aedt_agent.knowledge.sqlite_provider import SQLiteKnowledgeProvider
from aedt_agent.knowledge.build_sqlite import build_api_semantics_db


def test_sqlite_provider_search_returns_results(tmp_path):
    db_path = tmp_path / "test.sqlite"
    build_api_semantics_db(
        Path("knowledge/api_semantics/api_semantics.schema.sql"),
        Path("knowledge/api_semantics/api_semantics.seed.jsonl"),
        db_path,
    )
    provider = SQLiteKnowledgeProvider(db_path)

    results = provider.search_api("create_box", limit=5)

    assert len(results) >= 1
    assert results[0].fqname == "Hfss.modeler.create_box"


def test_sqlite_provider_lists_workflow_cases():
    provider = SQLiteKnowledgeProvider(
        db_path=Path("nonexistent.sqlite"),
        workflow_cases_dir=Path("knowledge/workflow_cases"),
        common_traps_dir=Path("knowledge/common_traps"),
    )

    cases = provider.list_workflow_cases()

    assert len(cases) >= 3
    assert any(c.case_id == "hfss_patch_antenna" for c in cases)


def test_sqlite_provider_lists_common_traps_filtered():
    provider = SQLiteKnowledgeProvider(
        db_path=Path("nonexistent.sqlite"),
        workflow_cases_dir=Path("knowledge/workflow_cases"),
        common_traps_dir=Path("knowledge/common_traps"),
    )

    traps = provider.list_common_traps(filter_ids=["airbox_too_small"])

    assert len(traps) >= 1
    assert traps[0].trap_id == "airbox_too_small"
```

- ☐ **步骤 2：运行测试验证失败**

运行：

```
python -m pytest tests/test_knowledge_provider.py -q
```

预期：FAIL。

- [ ] **步骤 3：创建 workflow cases 和 common traps YAML 文件**

按原计划和补丁中的格式，创建全部 10 个 workflow cases 和 7 个 common traps YAML 文件。具体内容参照原计划任务 5 和补丁任务 17 中的完整 YAML 定义。

- [ ] **步骤 4：实现 SQLite Provider 和构建工具**

实现 `sqlite_provider.py` 和 `build_sqlite.py`，具体代码与原计划任务 5 一致。

- [ ] **步骤 5：运行测试验证通过**

```
python -m pytest tests/test_knowledge_provider.py -q
```

预期：PASS，7 个测试通过。

- [ ] **步骤 6：Commit**

```
git add knowledge/workflow_cases knowledge/common_traps
src/aedt_agent/knowledge tests/test_knowledge_provider.py
git commit -m "feat: load workflow cases and traps with expanded data"
```

---

## 任务 6：实现节点 catalog 模型与注册表

**文件：**

- 创建：`src/aedt_agent/nodes/models.py`
- 创建：`src/aedt_agent/nodes/registry.py`
- 创建 8 个节点 YAML 文件
- 创建：`tests/test_node_registry.py`
- [ ] 按原计划任务 6 完整执行，代码和 YAML 内容不变。
- [ ] **Commit**

```
git add nodes/catalog src/aedt_agent/nodes tests/test_node_registry.py
git commit -m "feat: add stage a node catalog"
```

---

## 任务 7：实现 Benchmark 任务模型

**文件：**

- 创建：`src/aedt_agent/benchmark/models.py`
- 创建全部 30 个 Benchmark 任务 YAML 文件
- 创建：`tests/test_runner.py`（初始版本）
- ☐ **步骤 1：实现 BenchmarkTask 模型**

按原计划任务 7 中的 `models.py` 代码实现。

- ☐ **步骤 2：创建 30 个任务 YAML 文件**

按补丁任务 18 的分层定义创建：

L1（10 个单节点任务）：`L1_create_substrate`、`L1_create_conductor`、`L1_select_face`、`L1_create_wave_port`、`L1_create_lumped_port`、`L1_create_airbox`、`L1_assign_radiation_boundary`、`L1_create_setup`、`L1_create_sweep`、`L1_assign_material`

L2（10 个小工作流任务）：`L2_microstrip_line`、`L2_stripline`、`L2_cpw_line`、`L2_coaxial_cable`、`L2_waveguide_section`、`L2_patch_with_probe_feed`、`L2_dipole_antenna`、`L2_simple_filter`、`L2_cavity_resonator`、`L2_differential_pair`

L3（5 个完整闭环任务）：`L3_patch_antenna_sparameter`、`L3_waveguide_filter`、`L3_antenna_array`、`L3_vivaldi_antenna`、`L3_sparameter_export`

Trap（5 个反直觉陷阱任务）：`Trap_waveport_wrong_face`、`Trap_airbox_too_small`、`Trap_missing_ground`、`Trap_sweep_misses_freq`、`Trap_unit_mismatch`

每个任务 YAML 格式参照原计划任务 7 中的定义。

- ☐ **步骤 3：编写初始测试**

```
from pathlib import Path
from aedt_agent.benchmark.models import BenchmarkTask, load_tasks


def test_load_tasks_reads_30_yaml_files():
    tasks = load_tasks(Path("benchmarks/tasks"))

    assert len(tasks) == 30
    levels = {t.level for t in tasks}
    assert levels == {"L1", "L2", "L3", "Trap"}


def test_benchmark_task_exposes_expected_workflow():
```

```
    task =
BenchmarkTask.from_yaml(Path("benchmarks/tasks/L3_patch_antenna_sparameter.yaml
"))

    assert task.level == "L3"
    assert "create_port" in task.expected_workflow
```

- ☐ **步骤 4：Commit**

```
git add benchmarks/tasks src/aedt_agent/benchmark/models.py
tests/test_runner.py
git commit -m "feat: add 30 benchmark tasks with L1/L2/L3/Trap coverage"
```

---

## 任务 8：实现离线代码 Graders

**文件：**

- 创建：`src/aedt_agent/benchmark/graders.py`
- 创建：`tests/test_graders.py`
- ☐ 按原计划任务 8 完整执行，实现 `check_syntax`、`check_restricted_python`、`check_allowed_api_usage`。
- ☐ **Commit**

```
git add src/aedt_agent/benchmark/graders.py tests/test_graders.py
git commit -m "feat: add offline benchmark graders"
```

---

## 任务 9：实现 Semantic Lite 离线评估

**目标：** 在不连接 AEDT 的前提下，通过静态规则检查捕获步骤遗漏、前置依赖缺失、已知陷阱模式。这是 Go/No-Go 核心指标的计算基础。

**文件：**

- 创建：`src/aedt_agent/benchmark/semantic_lite.py`
- 创建：`tests/test_semantic_lite.py`
- ☐ **步骤 1：编写失败测试**

创建 `tests/test_semantic_lite.py`：

```
```

```
from aedt_agent.benchmark.semantic_lite import check_semantic_lite


def test_missing_step_detected():
    """代码中缺少 create_airbox 步骤应被检测"""
    from aedt_agent.benchmark.models import BenchmarkTask
    from aedt_agent.knowledge.models import CommonTrap

    task = BenchmarkTask.from_dict({
        "task_id": "L3_test", "level": "L3", "domain": "hfss",
        "requirement": "Create patch antenna",
        "allowed_nodes": ["create_substrate", "create_airbox"],
        "expected_workflow": ["create_substrate", "create_airbox"],
        "required_api_categories": ["geometry", "boundary"],
        "reference_script": "", "validation_script": "",
        "expected_outputs": [], "known_failure_modes": [], "grading": {},
    })

    code = "app.modeler.create_box([0,0,0],[1,1,1], name='substrate')"
    result = check_semantic_lite(code, task, api_semantics=[], traps=[])

    assert result.passed is False
    assert any("create_airbox" in v for v in result.violations)


def test_missing_dependency_detected():
    """create_port 前没有 select_face 应被检测"""
    code = "app.create_wave_port(assignment='face1', name='Port1')"
    result = check_semantic_lite(code, task=None, api_semantics=[], traps=[])

    assert result.passed is False
    assert any("get_object_faces" in v or "select_face" in v for v in
result.violations)


def test_valid_code_passes():
    """正确代码应通过"""
    code = """
app.modeler.create_box([0,0,0],["20mm","15mm","0.8mm"], name='substrate',
material='FR4_epoxy')
app.assign_material('substrate', 'FR4_epoxy')
app.modeler.create_box(["-5mm","-5mm","0.8mm"],["30mm","30mm","30mm"],
name='airbox')
app.assign_radiation_boundary_to_objects('airbox')
"""
    result = check_semantic_lite(code, task=None, api_semantics=[], traps=[])
    assert result.passed is True
```

- ☐ **步骤 2：运行测试验证失败**

```
python -m pytest tests/test_semantic_lite.py -q
```

- ☐ **步骤 3：实现 Semantic Lite 检查**

创建 src/aedt_agent/benchmark/semantic_lite.py：

```
from __future__ import annotations

from dataclasses import dataclass, field

from aedt_agent.benchmark.models import BenchmarkTask
from aedt_agent.knowledge.models import ApiSemantic, CommonTrap


STEP_API_MAP: dict[str, list[str]] = {
    "create_substrate": ["create_box", "assign_material"],
    "create_conductor_or_geometry_group": ["create_box", "create_rectangle",
"assign_material", "unite", "subtract"],
    "select_face": ["get_object_faces", "get_face_center"],
    "create_port": ["create_wave_port", "create_lumped_port"],
    "create_airbox": ["create_box"],
    "assign_boundary": ["assign_radiation_boundary", "assign_perfecte"],
    "create_setup": ["create_setup"],
    "create_sweep_or_export": ["create_linear_count_sweep",
"export_touchstone"],
}

DEFAULT_DEPENDENCIES: dict[str, list[str]] = {
    "create_wave_port": ["get_object_faces"],
    "create_lumped_port": ["get_object_faces"],
}


@dataclass(frozen=True)
class SemanticLiteResult:
    passed: bool
    violations: list[str] = field(default_factory=list)


def check_semantic_lite(
    code: str,
    task: BenchmarkTask | None,
    api_semantics: list[ApiSemantic],
    traps: list[CommonTrap],
) -> SemanticLiteResult:
    violations: list[str] = []

    # 1. 步骤覆盖检查
    if task is not None:
        for step in task.expected_workflow:
```

```
            step_apis = STEP_API_MAP.get(step, [])
            if step_apis and not any(api in code for api in step_apis):
                violations.append(f"missing_step: {step} (expected APIs:
{step_apis})")

    # 2. 前置依赖检查
    for target_api, required_apis in DEFAULT_DEPENDENCIES.items():
        if target_api in code:
            for req in required_apis:
                if req not in code:
                    violations.append(f"missing_dependency: {req} required
before {target_api}")

    return SemanticLiteResult(passed=len(violations) == 0,
violations=violations)
```

- ☐ **步骤 4：运行测试验证通过**

```
python -m pytest tests/test_semantic_lite.py -q
```

- ☐ **步骤 5：Commit**

```
git add src/aedt_agent/benchmark/semantic_lite.py tests/test_semantic_lite.py
git commit -m "feat: add semantic lite offline grader"
```

---

## 任务 10：实现 Group A/B/C 上下文构建器（含 Token 限制）

**目标：** 为三组实验构建不同深度的上下文，并确保检索上下文不超过 8k tokens。

**文件：**

- 创建：`src/aedt_agent/benchmark/context_builder.py`
- 创建：`tests/test_context_builder.py`
- ☐ **步骤 1：编写失败测试**

创建 `tests/test_context_builder.py`：

```
from pathlib import Path
from aedt_agent.benchmark.context_builder import build_context
from aedt_agent.benchmark.models import BenchmarkTask
from aedt_agent.knowledge.build_sqlite import build_api_semantics_db
from aedt_agent.knowledge.sqlite_provider import SQLiteKnowledgeProvider
from aedt_agent.nodes.registry import NodeRegistry
```

```
def _provider(tmp_path):
    db_path = tmp_path / "api_semantics.sqlite"
    build_api_semantics_db(
        Path("knowledge/api_semantics/api_semantics.schema.sql"),
        Path("knowledge/api_semantics/api_semantics.seed.jsonl"),
        db_path,
    )
    return SQLiteKnowledgeProvider(
        db_path,
        workflow_cases_dir=Path("knowledge/workflow_cases"),
        common_traps_dir=Path("knowledge/common_traps"),
    )


def test_group_a_context_contains_only_requirement(tmp_path):
    task =
BenchmarkTask.from_yaml(Path("benchmarks/tasks/L1_create_substrate.yaml"))
    context = build_context(group="A", task=task, provider=_provider(tmp_path),
registry=NodeRegistry.from_directory(Path("nodes/catalog")))
    assert task.requirement in context
    assert "API whitelist" not in context


def test_group_c_context_contains_nodes_api_and_traps(tmp_path):
    task =
BenchmarkTask.from_yaml(Path("benchmarks/tasks/L3_patch_antenna_sparameter.yaml
"))
    context = build_context(group="C", task=task, provider=_provider(tmp_path),
registry=NodeRegistry.from_directory(Path("nodes/catalog")))
    assert "API whitelist" in context
    assert "Common traps" in context
    assert "create_substrate" in context


def test_group_c_context_within_token_limit(tmp_path):
    task =
BenchmarkTask.from_yaml(Path("benchmarks/tasks/L3_patch_antenna_sparameter.yaml
"))
    context = build_context(group="C", task=task, provider=_provider(tmp_path),
registry=NodeRegistry.from_directory(Path("nodes/catalog")))
    estimated_tokens = len(context) // 4
    assert estimated_tokens <= 8000, f"Context too large: ~{estimated_tokens}
tokens"
```

- ☐ **步骤 2：实现上下文构建器**

创建 `src/aedt_agent/benchmark/context_builder.py`，包含 `build_context` 函数和 `_truncate_to_token_limit` 截断逻辑。Group A 只输出 requirement，Group B 追加 API 签名/docstring，Group C 追加节点白名单 + workflow cases + common traps。输出前检查 token 上限。

具体代码与原计划任务 9 的 `build_context` 一致，末尾追加 token 估算和截断：

```
def _estimate_tokens(text: str) -> int:
    return len(text) // 4

def _truncate_to_token_limit(text: str, max_tokens: int = 8000) -> str:
    if _estimate_tokens(text) <= max_tokens:
        return text
    return text[:max_tokens * 4] + "\n\n[Context truncated to fit token limit]"
```

在 `build_context` 返回前调用截断。

- ☐ **步骤 3：运行测试验证通过**

```
python -m pytest tests/test_context_builder.py -q
```

- ☐ **步骤 4：Commit**

```
git add src/aedt_agent/benchmark/context_builder.py
tests/test_context_builder.py
git commit -m "feat: build benchmark prompt contexts with token limit"
```

---

## 任务 11：实现 LLM 代码生成接口

**目标：** 抽象 LLM 调用接口，支持 File/OpenAI/Anthropic 多后端，使 Group A/B/C 可以真正用 LLM 生成代码。

**文件：**

- 创建：`src/aedt_agent/benchmark/generator.py`
- 创建：`src/aedt_agent/benchmark/prompt_templates.py`
- 创建：`tests/test_generator.py`
- ☐ **步骤 1：编写失败测试**

创建 `tests/test_generator.py`：

```
from aedt_agent.benchmark.generator import CodeGenerator, FileGenerator,
DefaultCodeGenerator, create_generator_from_env
from aedt_agent.benchmark.prompt_templates import build_prompt


def test_build_prompt_group_a_contains_only_requirement():
```

```
    prompt = build_prompt(group="A", requirement="Create a 2.4GHz patch
antenna", context="")
    assert "2.4GHz patch antenna" in prompt
    assert "API whitelist" not in prompt


def test_build_prompt_group_c_contains_full_context():
    prompt = build_prompt(group="C", requirement="Create a 2.4GHz patch
antenna", context="API whitelist:\n- Hfss.modeler.create_box\n\nCommon
traps:\n- missing_ground_plane")
    assert "API whitelist" in prompt
    assert "missing_ground_plane" in prompt


def test_file_generator_reads_from_disk():
    import tempfile
    from pathlib import Path
    with tempfile.TemporaryDirectory() as tmp:
        code_path = Path(tmp) / "test_code.py"
        code_path.write_text("app.modeler.create_box([0,0,0],[1,1,1])",
encoding="utf-8")
        gen = FileGenerator(base_dir=Path(tmp))
        code = gen.generate(context="any", filename="test_code.py")
        assert "create_box" in code


def test_default_generator_raises_not_implemented():
    gen = DefaultCodeGenerator()
    try:
        gen.generate(context="test")
        assert False, "Should have raised NotImplementedError"
    except NotImplementedError:
        pass
```

- ☐ **步骤 2：实现 prompt 模板和代码生成器**

`prompt_templates.py`：三组 prompt 策略，Group A 裸任务、Group B 加 API 签名、Group C 加白名单+案例+陷阱。

`generator.py`：定义 `CodeGenerator` Protocol，实现 `DefaultCodeGenerator`（抛 NotImplementedError）、`FileGenerator`（从磁盘读取）、`OpenAIGenerator`、`AnthropicGenerator`，以及 `create_generator_from_env()` 从环境变量选择后端。

具体代码参照补丁任务 15。

- ☐ **步骤 3：运行测试验证通过**

```
python -m pytest tests/test_generator.py -q
```

- ☐ **步骤 4：Commit**

```
git add src/aedt_agent/benchmark/generator.py
src/aedt_agent/benchmark/prompt_templates.py tests/test_generator.py
git commit -m "feat: add llm code generator with multi-backend support"
```

---

## 任务 12：扩展 API 语义库到 Top 50

**目标：** 从 5 条 seed 扩展到 Top 50，覆盖 8 个节点的 api_whitelist。

**文件：**

- 创建：`src/aedt_agent/knowledge/extract_api_semantics.py`
- 修改：`knowledge/api_semantics/api_semantics.seed.jsonl`（扩展到 50 条）
- 创建：`tests/test_api_semantics_coverage.py`
- ☐ **步骤 1：编写覆盖率测试**

```
from pathlib import Path
from aedt_agent.knowledge.sqlite_provider import SQLiteKnowledgeProvider
from aedt_agent.knowledge.build_sqlite import build_api_semantics_db
from aedt_agent.nodes.registry import NodeRegistry


def test_api_semantics_covers_node_whitelists(tmp_path):
    db_path = tmp_path / "api_semantics.sqlite"
    build_api_semantics_db(
        Path("knowledge/api_semantics/api_semantics.schema.sql"),
        Path("knowledge/api_semantics/api_semantics.seed.jsonl"),
        db_path,
    )
    provider = SQLiteKnowledgeProvider(db_path)
    registry = NodeRegistry.from_directory(Path("nodes/catalog"))

    all_whitelist_apis: set[str] = set()
    for node in registry.list_nodes():
        all_whitelist_apis.update(node.api_whitelist)

    uncovered: set[str] = set()
    for api in all_whitelist_apis:
        results = provider.search_api(api, limit=1)
        if not results or results[0].fqname != api:
            uncovered.add(api)

    assert len(uncovered) == 0, f"APIs not in semantics library: {uncovered}"
```

```
def test_api_semantics_has_at_least_50_entries(tmp_path):
    db_path = tmp_path / "api_semantics.sqlite"
    build_api_semantics_db(
        Path("knowledge/api_semantics/api_semantics.schema.sql"),
        Path("knowledge/api_semantics/api_semantics.seed.jsonl"),
        db_path,
    )
    provider = SQLiteKnowledgeProvider(db_path)
    categories = ["geometry", "material", "boundary", "excitation", "setup",
"postprocess"]
    total = sum(len(provider.search_api(cat, limit=100)) for cat in categories)
    assert total >= 50, f"Only {total} API entries, need at least 50"
```

- ☐ **步骤 2：实现自动抽取脚本**

创建 `extract_api_semantics.py`，定义 `TOP_50_APIS` 清单（覆盖 8 个节点的 api_whitelist + benchmark 任务所需的 50 个核心 API），从 pyaedt 源码提取 docstring。

具体代码参照补丁任务 16。

- ☐ **步骤 3：运行抽取脚本并人工精标**

```
python -m aedt_agent.knowledge.extract_api_semantics --pyaedt-src
D:/code/pyaedt-src/src/ansys/aedt/core --output
knowledge/api_semantics/api_semantics.seed.jsonl --top-50
```

对 `confidence: template` 的条目人工补充签名、约束、常见错误，升级为 `manual`。

- ☐ **步骤 4：运行覆盖率测试**

```
python -m pytest tests/test_api_semantics_coverage.py -q
```

- ☐ **步骤 5：Commit**

```
git add knowledge/api_semantics/api_semantics.seed.jsonl
src/aedt_agent/knowledge/extract_api_semantics.py
tests/test_api_semantics_coverage.py
git commit -m "feat: expand api semantics to top 50 with auto-extraction"
```

---

## 任务 13：实现 Go/No-Go 指标计算

**目标：** 在 runner 输出上计算 8 项量化指标和 Go 条件判定。

**文件：**

- 创建：`src/aedt_agent/benchmark/go_nogo.py`
- 创建：`tests/test_go_nogo.py`
- ☐ **步骤 1：编写失败测试**

```
from aedt_agent.benchmark.go_nogo import compute_go_nogo


def test_compute_go_nogo_passes_when_metrics_met():
    report = {
        "tasks": {
            "L1_create_substrate": {
                "A": {"syntax_pass": True, "api_pass": True,
"semantic_lite_pass": False},
                "B": {"syntax_pass": True, "api_pass": True,
"semantic_lite_pass": True},
                "C": {"syntax_pass": True, "api_pass": True,
"semantic_lite_pass": True},
            },
        }
    }
    result = compute_go_nogo(report)
    assert result["metrics"]["semantic_pass_rate_c"] >= 0.70
```

- ☐ **步骤 2：实现 compute_go_nogo**

计算 api_pass_rate_c、semantic_pass_rate_b/c、semantic_lift、trap_capture_rate，判定 Go 条件。

具体代码参照补丁任务 14。

- ☐ **步骤 3：运行测试验证通过**

```
python -m pytest tests/test_go_nogo.py -q
```

- ☐ **步骤 4：Commit**

```
git add src/aedt_agent/benchmark/go_nogo.py tests/test_go_nogo.py
git commit -m "feat: add go/nogo metrics computation"
```

---

## 任务 14：实现 Repair Loop 记录

**目标：** 记录代码生成和修复轮次，支持"首次失败后附加 traceback 再生成"。

**文件：**

- 创建：`src/aedt_agent/benchmark/repair.py`
- 创建：`tests/test_repair.py`
- ☐ **步骤 1：编写失败测试**

```
from aedt_agent.benchmark.repair import RepairRecord, run_with_repair


def test_repair_record_tracks_rounds():
    record = RepairRecord(task_id="test", group="C")
    record.add_round(round_num=1, code="bad code", passed=False,
error="SyntaxError")
    record.add_round(round_num=2, code="fixed code", passed=True, error="")
    assert record.total_rounds == 2
    assert record.success is True
    assert record.repair_count == 1
```

- ☐ **步骤 2：实现 RepairRecord 和 run_with_repair**

`RepairRecord` 记录轮次和修复次数。`run_with_repair` 最多 2 轮，首轮失败后追加 traceback 到 context。

具体代码参照补丁任务 21。

- ☐ **步骤 3：运行测试验证通过**

```
python -m pytest tests/test_repair.py -q
```

- ☐ **步骤 4：Commit**

```
git add src/aedt_agent/benchmark/repair.py tests/test_repair.py
git commit -m "feat: add repair loop with round tracking"
```

---

## 任务 15：实现 candidate-ready 节点清单汇总

**目标：** 根据 benchmark 判卷结果，列出达到 candidate-ready 条件的节点。

**文件：**

- 创建：`src/aedt_agent/benchmark/node_readiness.py`
- 创建：`tests/test_node_readiness.py`
- ☐ 按补丁任务 22 实现 `compute_node_readiness`，准入条件：3 个 Benchmark 覆盖、两轮成功率 >= 85%、Semantic pass >= 70%。

- [ ] **Commit**

```
git add src/aedt_agent/benchmark/node_readiness.py tests/test_node_readiness.py
git commit -m "feat: add candidate-ready node readiness evaluation"
```

---

## 任务 16：实现完整 Benchmark Runner

**目标：** 整合 graders + semantic_lite + go_nogo + repair，输出含完整五层评估和 Go/No-Go 判定的报告。

**文件：**

- 创建：`src/aedt_agent/benchmark/runner.py`
- 修改：`tests/test_runner.py`
- [ ] **步骤 1：编写失败测试**

```
from pathlib import Path
from aedt_agent.benchmark.runner import run_offline_benchmark


def test_run_offline_benchmark_reports_group_results(tmp_path):
    report = run_offline_benchmark(
        tasks_dir=Path("benchmarks/tasks"),
        generated_dir=Path("benchmarks/generated"),
        node_catalog_dir=Path("nodes/catalog"),
        report_path=tmp_path / "report.json",
    )
    assert "L1_create_substrate" in report["tasks"]
    assert "go_nogo" in report
```

- [ ] **步骤 2：实现 runner**

`run_offline_benchmark` 遍历所有任务和三组，对每个代码文件执行：

1. `check_syntax`
2. `check_restricted_python`
3. `check_allowed_api_usage`
4. `check_semantic_lite`
5. 记录结果
6. 调用 `compute_go_nogo` 计算指标
7. 调用 `compute_node_readiness` 计算 candidate-ready 清单
8. 输出 JSON 报告

- [ ] **步骤 3：运行测试验证通过**

```
python -m pytest tests/test_runner.py -q
```

- [ ] **步骤 4：Commit**

```
git add src/aedt_agent/benchmark/runner.py tests/test_runner.py
git commit -m "feat: add full benchmark runner with go/nogo and node readiness"
```

---

## 任务 17：实现 CLI 入口

**文件：**

- 创建：`src/aedt_agent/cli.py`
- [ ] 按原计划任务 11 实现 `build-db` 和 `run-benchmark` 两个子命令。
- [ ] **Commit**

```
git add src/aedt_agent/cli.py
git commit -m "feat: add stage a benchmark cli"
```

---

## 任务 18：创建 Reference Scripts

**目标：** 为每个 benchmark 任务编写参考脚本，优先从 pyaedt 官方 examples 改写。

**文件：**

- 在 `benchmarks/reference_scripts/` 下创建每个任务对应的 `.py` 文件
- [ ] **步骤 1：编写 L1 参考脚本**

每个 L1 脚本只调用一个节点的 API：

```
# benchmarks/reference_scripts/L1_create_substrate.py
app.modeler.create_box(
    origin=["0mm", "0mm", "0mm"],
    sizes=["20mm", "15mm", "0.8mm"],
    name="substrate",
    material="FR4_epoxy",
)
app.assign_material("substrate", "FR4_epoxy")
```

- [ ] **步骤 2：编写 L2/L3/Trap 参考脚本**

L2 覆盖小工作流，L3 是完整闭环，Trap 展示错误和正确写法对比。

- ☐ **步骤 3：Commit**

```
git add benchmarks/reference_scripts
git commit -m "feat: add reference scripts for benchmark tasks"
```

---

## 任务 19：编写 Validation Script 规范

**目标：** 为 Stage B 定义 validation script 接口规范。

**文件：**

- 创建：`docs/validation-script-spec.md`

- ☐ **步骤 1：编写规范文档**

定义接口签名（`validate(session_id, project_id, design_id) -> dict`）、规则（每条检查至少一条 assert、Stage A 允许 mock、Trap 任务必须检测静默失败、不允许修改 AEDT 状态）。

具体内容参照补丁任务 19。

- ☐ **步骤 2：Commit**

```
git add docs/validation-script-spec.md
git commit -m "docs: add validation script specification"
```

---

## 任务 20：生成示例报告并记录验收方式

**文件：**

- 创建：`benchmarks/reports/stage_a_sample_report.json`

- ☐ **步骤 1：构建本地 SQLite 数据库**

```
python -m aedt_agent.cli build-db --schema
knowledge/api_semantics/api_semantics.schema.sql --seed
knowledge/api_semantics/api_semantics.seed.jsonl --db
knowledge/api_semantics/api_semantics.sqlite
```

- ☐ **步骤 2：运行离线 Benchmark**

```
python -m aedt_agent.cli run-benchmark --tasks benchmarks/tasks --generated
benchmarks/generated --nodes nodes/catalog --report
```

```
benchmarks/reports/stage_a_sample_report.json
```

- [ ] **步骤 3：检查报告**

报告必须包含 `go_nogo` 字段和 `node_readiness` 字段。

- [ ] **步骤 4：运行完整测试**

```
python -m pytest -q
```

预期：全部测试通过。

- [ ] **步骤 5：Commit**

```
git add knowledge/api_semantics/api_semantics.sqlite
benchmarks/reports/stage_a_sample_report.json
git commit -m "test: add sample stage a benchmark report with go/nogo"
```

---

# 自检

规格覆盖度：

- Top 50 API 语义：任务 12 覆盖
- 10 个 workflow cases：任务 5 覆盖
- 7 个 common traps：任务 5 覆盖
- 8 个节点 catalog：任务 6 覆盖
- 30 个 Benchmark 任务：任务 7 覆盖
- 五层评估（syntax/api/security/semantic-lite/repair）：任务 8/9/14 覆盖
- Go/No-Go 指标计算：任务 13 覆盖
- LLM 代码生成接口：任务 11 覆盖
- Token 限制检查：任务 10 覆盖
- candidate-ready 节点清单：任务 15 覆盖
- Validation script 规范：任务 19 覆盖
- Reference scripts：任务 18 覆盖

未完成标记扫描：

- 每个步骤都有具体文件、代码或命令。

类型一致性：

- `ApiSemantic`、`WorkflowCase`、`CommonTrap` 在 Provider 和测试中字段一致。
- `NodeDefinition` 与节点 YAML 字段一致。
- `BenchmarkTask` 与任务 YAML 字段一致。
- `SemanticLiteResult` 在 graders/runner/go_nogo 中一致。

- `CodeGenerator` Protocol 在 generator/context_builder/runner 中一致。